

테순라: 자율 추종 카트 시스템

순천향대학교 사물인터넷학과

버전 <2.0.3>

<2026/06/16>

팀명: 테슨라

제정 및 개정 이력

버전	개정 내용	작성자	승인자	적용 날짜
1.0	최초 생성	권범철	권범철	2025.10.29
1.0.1	1번 개요 작성 - 목적, 개발배경, 기존 공항 카트 분석 및 기대효과 추가	권범철	권범철	2025.10.29
1.0.2	2번 요구 사항 정의 추가 - 하드웨어, 소프트웨어, 성능 요구사항 추가	권범철	권범철	2025.11.04
1.0.3	3번 요구 기능 설계 추가 - 소프트웨어 기능 설계부분 추가 (AI, WIFI부분)	이지수	이지수	2025.11.18
1.0.4	4번 요구 기능 설계 추가 -하드웨어 기능 설계 부분 추가 (마스터, 슬레이브 부분)	권범철	이지수	2025.11.23
1.0.5	5번 요약 및 결론 추가	이지수	이지수	2025.11.31
1.0.6	최종 수정	김준형	권범철	2025.12.02
2.0.1	4.1.2 테스트 케이스 수정	권범철	권범철	2026.06.15
2.0.2	3. 하드웨어 기능설계, 4.1.1, 4.2.1, 전체 아키텍처 설명	김준형	김준형	2026.06.15

2.0.3	1.개요, 2.1.2 슬레이브 카트 기능 요구사항, 2.2 소프트웨어 기능 요구사항, 3.3 성능 요구사항, 4.1.2 ODOMETRY + IMU 기능 테스트,, 4.2.2 슬레이브 시스템 아키텍쳐 설명	이지수	이지수	2026.06.17
2.0.3	2.1.3 마스터 카트 요구사항, 2.3 마스터 성능 요구사항, 3.2 소프트웨어 기능 설계, 4.1.2 SLAM 기능 테스트, 4.2.2 마스터 시스템 아키텍쳐 설명	정다혁	정다혁	2026.06.17

목 차

1 개요

1.1 목적 5

1.2 개발배경 5

1.3 기존 공항 카트 분석 5

1.4 시스템 개요 5

1.5 기대 효과 5

1.6 제약 사항 5

1.7 개발 일정 5

2 요구사항 정의 6

2.1 하드웨어 기능 요구사항 6

2.2 소프트웨어 기능 요구사항 6

2.3 (마스터) 성능 요구사항 6

3 요구 기능 설계 7

3.1 하드웨어 기능 설계 7

3.1.1 H-F-01: 메인 제어유닛(MCU)은 ROS2 구동을 위한 연산성능을 보장해야 한다 7

3.1.2 H-F-02: 정밀 주행 제어를 위한 피드백 구동 시스템을 구축해야 한다. 7

3.1.3 H-F-03: 주변 환경 인식 및 SLAM을 위한 2D LiDAR 센서를 장착해야 한다 7

3.1.4 H-F-04: 안정적인 시스템 구동을 위한 전원 분배 시스템(PDU)을 갖춰야 한다 7

3.1.5 H-F-05: 화물 적재 및 주행 안정성을 고려한 차체 프레임을 설계해야 한다 7

3.1.6 상태 모니터링을 위한 디스플레이 연결 인터페이스를 지원해야 한다. 7

3.1.7 최종 하드웨어 결과물 7

3.2 소프트웨어 기능 설계 8

3.2.1 S-M-01: 마스터 카트는 ROS2 Nav2를 사용하여 경로를 생성하고 자율주행을 수행해야 한다 8

3.2.2 S-M-02: 마스터는 장애물 및 사람 감지 시 즉시 정지·회피를 수행해야 한다. 8

3.2.3 S-M-03:마스터-슬레이브 간 제어 및 상태 정보를 무선으로 양방향 통신해야 한다.

8

3.2.4 S-M-04:자신의 현재 상태(위치, 속도, 상태 플래그)를 슬레이브 카트로 주기적(예: 20Hz)이며 안정적으로 전송할 수 있어야 한다. 8

3.2.5 S-M-05: ROS2 네임스페이스 기반 다중 슬레이브 관리 기능을 제공해야 한다. 8

3.2.6 S-M-06: 마스터는 RViz를 통해 주행 상태를 시각화해야 한다.. 8

3.2.7 S-M-07:마스터는 좌표 반환을 통해 센서 데이터와 위치 정보를 통합해야 한다. 8

3.2.8 S-M-08: 승인되지 않은 장치는 제어망에 접근할 수 없어야 한다. 8

3.2.9 S-M-09:마스터는 엔코더 오도메트리로 이동거리와 방향을 계산해야 한다. 8

3.2.10 S-M-10:마스터는 GPU(CUDA)로 YOLO 객체 인식을 실시간 처리해야 한다. 8

3.2.11 S-S-01: 슬레이브는 마스터가 전송하는 속도/경로 명령을 수신하여 주행 제어를 수행해야 한다. 8

3.2.12 S-S-02: 슬레이브는 마스터와의 추종 거리를 목표값(예: 1m±허용 오차)으로 유지해야 한다. 8

3.2.13 S-S-03: 슬레이브는 전방 장애물 감지 시 감속·정지하고 장애물 이벤트를 마스터에 보고해야 한다. 8

3.2.14 S-S-04: 슬레이브는 마스터와의 통신 타임아웃 발생 시 자동으로 안전 정지 모드로 전환해야 한다. 8

3.2.15 S-S-05: 슬레이브는 배터리·통신·동작 상태를 주기적으로 마스터에 전송해야 한다.

8

3.2.16 S-S-06: 슬레이브는 비상 정지 입력(물리 스위치 또는 UI 버튼 등)과 운용 모드를 소프트웨어 상태로 관리해야 한다. 8

3.2.17 S-S-07:슬레이브는 WiFi 통신이 끊기면 카메라 ArUco 마커 추종으로 자동 전환해야 한다 8

3.2.18 S-N-01: 주행 동기화를 위한 실시간 제어 데이터를 멀티 캐스팅해야 한다.. 8

3.2.19 S-N-02: 마스터-슬레이브 간 양방향 상태 정보를 동기화해야 한다. 8

3.2.20 S-N-03: 통신 연결 상태 감지 및 타임아웃 처리를 수행해야 한다. 8

3.2.21 S-N-04: 긴급 상황 발생 시 최우선 순위로 정지 명령을 전파해야 한다. 8

3.2.22 S-N-05: ROS2 네임스페이스 기반 다중 슬레이브 식별 체계를 갖춰야 한다. 8

3.2.23 S-N-06: 마스터와 슬레이브가 멀어졌을 시 마스터는 대기해야 한다. 8

3.2.24 S-N-07: Wi-Fi 기반 제어 네트워크 인프라를 구축해야 한다. 8

3.3 성능 요구사항 설계 9

3.3.1 PR-1: 기본 하중 최소 1kg 이상의 화물을 적재한 상태에서 정상 주행 가능해야 한다 9

3.3.2 PR-2: 안전한 추종 주행을 위한 적정 주행 속도를 유지해야 한다. 9

3.3.3 PR-3: 마스터-슬레이브 간 목표 추종 거리 및 오차 범위를 준수해야 한다9

3.3.4 PR-4: 무선 통신 지연(Latency)을 최소화하여 실시간성을 확보해야 한다9
3.3.5 PR-5: 장애물 감지 및 긴급 제동 성능을 만족해야 한다. 9
3.3.6 PR-6: 시스템 운용 시간 및 전력 효율성을 확보해야 한다. 9
4 테스트 10
4.1 기능 테스트 10
4.1.1 테스트 주의사항 및 기타 고려사항 10
4.1.2 테스트 케이스 10
4.2 시스템 테스트 11
4.2.1 테스트 주의사항 및 고려사항 11
4.2.2 테스트 케이스 11
5 요약 및 결론 12
부록 1: 참고 문헌 13
부록 2: 용어 14

1 개요

1.1 목적

- 본 프로젝트는 공항, 터미널, 쇼핑몰, 병원 등 넓은 실내 공간에서 사용자의 수하물 또는 물품 운반 부담을 줄이기 위한 자율 추종 카트 시스템 개발을 목적으로 함.
- 공항 환경의 경우 게이트 간 이동 거리가 길고, 무거운 수하물을 직접 운반해야 하는 부담 존재.
- 특히 노약자, 어린이 동반 승객, 보행 취약 계층의 이동 불편 증가.
- 목표: AI 기반 무선 추종 트랙커 개발

- 목적지까지 효율적•안전한 물품 배송
- 혼잡 환경에서도 안정적 운행 가능
- 사용자의 수하물 운반 부담 감소
- 공항 및 대형 실내 공간에서의 이동 편의성 향상
- Master-Slave 구조 기반 저비용 자율 추종 시스템 구현
- 다수 Slave 카트 확장 운용 가능 구조 확보
- 향후 무인 물류 운반 및 여객 보조 서비스로 확장 가능한 기반 구축

1.2 개발 배경

- 공항 이용객 증가 및 대형 실내 시설 확대로 인해 수하물 운반 보조 시스템 필요성 증가.
- 기존 공항 카트의 한계
  - 적재 무게 및 크기 제한
  - 인파 많은 환경에서 운행 편의성 및 안전성 저하 가능
  - 사용자가 직접 밀고 장거리 이동 시 피로도 증가
- 인프라 구축 없이 적용 가능한 무선 추종형 시스템의 필요성
- IoT 및 머신러닝 기술 융합형 운송 자동화 솔루션으로서의 가치
- 상용 공항 카트 로봇은 고가의 센서와 통합 시스템을 기반으로 구성되어 대량 도입 시 구축 비용 부담 큼.

1.3 기존 공항 카트 분석

- 기존 카트 문제점
  - 무거운 수하물을 직접 밀어야 하는 부담 발생
  - 게이트 간 장거리 이동 시 사용자 피로도 증가

- 노약자, 어린이 동반 승객 등 교통 약자에게 불편
- 혼잡한 공간에서 충돌 위험 및 이동 불편 발생
- 다수 카트의 효율적 관리 어려움

1.4 시스템 개요

- 본 시스템의 명칭은 테순라(TESUNLA)이며, Master-Slave 추종 기반 자율주행 카트 시스템임.
- 시스템은 크게 Master 카트와 Slave 카트로 구성.
    - **Master 카트**
        - 목적지까지 자율주행 수행. 자신의 위치 및 경로 데이터를 Slave 카트에 실시간 UDP 송신.
        - NVIDIA Jetson Orin Nano 기반 연산
        - 2D LiDAR 기반 SLAM 및 지도 생성
        - Nav2 기반 목적지 자율주행
        - IMU 및 Odometry 기반 위치 추정
        - YOLOv8 nano 기반 객체 인식
        - UDP/Wi-Fi 기반 위치 데이터 송신
    - **Slave 카트**
        - Master로부터 수신한 위치 데이터를 기반으로 추종 주행 수행.

- Raspberry Pi 4 기반 제어
- IMU 및 Odometry 기반 자세·위치 추정
- Master 위치 값과 비교하여 주행 제어
- 카메라 기반 ArUco 마커 추종
- 초음파 센서 기반 장애물 인식 및 정지
- 적재물 운반을 위한 저비용 구동 구조

1.5 기대 효과

- **기술적 효과**
  - LiDAR SLAM 기반 실내 자율주행 구현
  - Nav2 기반 목적지 이동 기능 구현
  - Master-Slave 기반 실시간 무선 추종 구조 구현
  - ArUco 마커, IMU, Odometry 기반 보조 추종 기능 확보
  - 초음파 센서 기반 장애물 감지 및 안전 정지 기능 구현
  - 다수 Slave 카트 연동을 통한 군집 주행 구조 확장 가능
- **경제적 효과**
  - 고가 센서 및 연산 장치의 중복 탑재 최소화
  - Master 1대와 저비용 Slave 다수 운용 구조를 통한 구축 비용 절감
  - Slave 제작 비용 절감을 통한 대량 도입 가능성 증가

- Master 제작 비용 약 5,513,500원, Slave 제작 비용 약 1,997,800원
- Slave는 Master 대비 약 64% 비용 절감 가능

- **사용자 효과**
  - 수하물 운반 부담 감소
  - 공항, 터미널, 병원, 쇼핑몰 등 대형 공간에서 이동 편의성 향상
  - 노약자 및 보행 취약 계층의 이동 보조 가능
  - 혼잡 환경에서 보다 편리한 물품 운반 가능
  - 인건비 절감 및 24시간 물류 비품 운반 가능
- **확장성**
  - Master 1대에 여러 Slave 카트 연결 가능
  - 수하물 운반량 및 물류 운반량 확장 가능
  - 물류 창고, 백화점, 호텔, 병원, 공공시설 등 다양한 공간 적용 가능
  - 향후 사람 회피 기능 고도화, 군집 주행, 자율 복귀 및 무인 운영 시스템으로 확장 가능

1.6 제약 사항

1.6.1

1.6.2 운용 환경(일반 실내/체육관)

- 공항, 학교, 터미널, 쇼핑몰 등 평탄한 실내 환경 중심 설계

- 경사로, 문턱, 요철이 큰 노면에서 주행 안정성 저하 가능
- 유리, 금속, 반사 재질이 많은 환경에서 센서 인식 성능 저하 가능

1.6.3 통신 제약

- Master-Slave 간 위치 데이터 송수신은 Wi-Fi/UDP 기반
- 통신 지연 또는 끊김 발생 시 추종 안정성 저하 가능
- 통신 불안정 상황 보완을 위한 ArUco 마커 기반 보조 추종 기능 적용

1.6.4 인식 제약

- LiDAR SLAM 성능은 실내 구조와 맵 품질에 영향 받음
- YOLO 기반 객체 인식은 조명, 거리, 카메라 화각에 따라 정확도 변화
- ArUco 마커는 가림 현상 또는 카메라 시야 이탈 시 인식 어려움
- 초음파 센서는 근거리 장애물 감지에는 유용하나 복잡한 형상 인식에는 한계 존재

1.6.5 안전 제약

- 실제 공항 서비스 적용 전 추가 안전 검증 필요
- 군중 밀집 환경에서 사람 회피, 속도 제어, 비상 정지 기능 고도화 필요
- 다수 Slave 운용 시 충돌 방지 및 경로 간섭 방지 알고리즘 보완 필요

1.7 개발 일정

주제	추진 내용	추진 일정														
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
하드웨어	마스터 제작 (40시간)															
	슬레이브 제작 (40시간)															
	오류 체크 (16시간)															
소프트웨어	마스터 ROS 구축(39시간)															
	슬레이브 ROS 구축(20시간)															
	마스터-슬레이브 통신 구축(60시간)															
통합	HW-SW 통합 (35시간)															

테스트 및 검증	하드웨어 센서 테스트(8시간)	
	딥러닝 오차 보정(28시간)	
	결과 보고서 작성(30시간)	

2 요구사항 정의

2.1 하드웨어 기능 요구사항

2.1.1 공동 요구사항

번호	요구 사항
H-01	프레임은 설정된 최대 적재 하중(예: 20kg)을 견딜 수 있는 충분한 기계적 강성을 가져야 한다.
H-02	구동계(모터)는 최대 하중 적재 시에도 설정된 속도(예: 1.0 m/s)로 안정적인 주행이 가능한 토크를 제공해야 한다.
H-03	구동계 모터는 정밀한 속도 및 위치 제어를 위해 인코더(Encoder)를 반드시 포함해야 한다.

H-04	전원 시스템은 모든 전자 장치 및 구동계를 최소 1시간(목표치) 이상 구동할 수 있는 충분한 배터리 용량을 가져야 한다.
H-05	전원 시스템은 메인 컴퓨터, 센서, 모터 드라이버 등 각 장치에 맞는 안정적인 DC 전압(예: 5V, 12V, 24V)을 공급해야 한다.
H-06	각 카트는 작업자가 즉시 시스템 전원을 물리적으로 차단할 수 있는 비상 정지(E-Stop) 스위치를 탑재해야 한다.

2.1.2

번호	요구 사항
H-S-01	슬레이브 카트는 Raspberry Pi 4를 메인 제어 유닛으로 사용하여 센서 데이터 수집, 추종 제어, 모터 명령 생성을 수행한다.
H-S-02	슬레이브 카트는 Wi-Fi AP를 통해 마스터 카트로부터 UDP 기반 위치 및 경로 데이터를 수신한다.
H-S-03	슬레이브 카트는 수신한 마스터 위치 데이터와 자신의 IMU/Odometry 데이터를 비교하여 추종 주행을 수행한다.
H-S-04	슬레이브 카트는 EBIMU 센서를 통해 yaw 값을 측정하고, 마스터 추종 시 방향 보정에 활용한다.
H-S-05	슬레이브 카트는 USB Webcam을 탑재하여 ArUco 마커의 x, y, z 좌표 벡터를 인식한다.
H-S-06	Wi-Fi 통신이 불안정하거나 끊길 경우, 슬레이브 카트는 카메라 기반 ArUco 마커 추종 방식으로 보조 추종을 수행한다.
H-S-07	슬레이브 카트는 초음파 센서 4개를 탑재해 전방 및 주변의 근거리 장애물을 감지한다.

H-S-08	초음파 센서가 50 cm 이내의 장애물을 감지할 경우, 슬레이브 카트는 정지 신호를 생성해 모터 구동을 중단한다.
H-S-09	슬레이브 카트는 MD400T 모터 드라이버를 통해 좌우 모터 2개를 제어한다.
H-S-10	슬레이브 카트는 Raspberry Pi 4에서 생성한 `/cmd_vel` 명령을 MD400T 모터 드라이버로 전달하여 주행 속도와 회전 방향을 제어한다.
H-S-11	슬레이브 카트는 택트 스위치 또는 Drive Switch를 통해 사용자가 start/stop 신호를 직접 입력할 수 있도록 구성한다.
H-S-12	슬레이브 카트는 5V를 Raspberry Pi 4에 공급하고, 24V를 모터 드라이버 및 구동부에 공급한다.
H-S-13	슬레이브 카트는 배터리 1개와 Power Switch 1개를 기반으로 독립 전원 구동이 가능하도록 구성한다.
H-S-14	슬레이브 카트는 마스터와 일정 추종 거리를 유지하며, 거리 오차 발생 시 속도 및 방향을 보정한다.
H-S-15	슬레이브 카트는 수하물 적재 상태에서도 구조적 흔들림을 최소화할 수 있도록 적재부와 구동부를 안정적으로 고정한다.

2.1.3 슬레이브 카트 기능 요구사항마스터 카트 요구사항

번호	요구 사항
H-M-01	메인 제어 유닛(MCU)은 SLAM/AMCL 및 Dijkstra 경로 탐색 알고리즘을 실시간으로 연산할 수 있는 충분한 컴퓨팅 성능을 갖춰야 한다.

H-M-02	공항 환경 지도 생성 및 현재 위치 추정을 위해 360도 전방위 스캔이 가능한 LiDAR 센서를 탑재해야 한다.
H-M-03	주행 중 발생하는 Odometry(주행기록계) 오차를 보정하고 정확한 방향(Heading)을 파악하기 위해 IMU 센서를 탑재해야 한다.
H-M-04	승객이 목적지(게이트)를 시각적으로 선택하고 출발 명령을 내릴 수 있는 터치스크린 HMI(디스플레이)를 제공해야 한다.
H-M-05	자신의 현재 상태(위치, 속도, 상태 플래그)를 슬레이브 카트로 주기적(예: 20 Hz)이며 안정적으로 전송할 수 있어야 한다.
H-M-06	사람을 인식하고 거리를 측정하기 위해 깊이(Depth) 카메라를 탑재해야 한다
H-M-07	정밀 주행 제어를 위해 엔코더 피드백 기반 모터 구동 시스템을 갖춰야 한다.
H-M-08	고연산 처리(SLAM, 객체 인식)를 가속하기 위한 GPU를 탑재해야 한다.
H-M-09	안정적인 전원 공급을 위한 전원 분배 시스템과 비상 정지 장치를 갖춰야 한다.

2.2 소프트웨어 기능 요구사항

번호	요구 사항
S-01	시스템은 ROS2 기반으로 마스터 카트와 슬레이브 카트의 센서 입력, 주행 제어, 통신 기능을 노드 단위로 구성한다.
S-02	마스터 카트는 2D LiDAR의 /scan 데이터를 이용하여 Cartographer SLAM 기반 지도 생성을 수행한다.

S-03	마스터 카트는 생성된 Occupancy Grid Map을 기반으로 Nav2 자율주행을 수행한다.
S-04	마스터 카트는 목표 지점 입력 시 현재 위치에서 목적지까지의 경로를 생성하고 `/cmd_vel` 명령을 출력한다.
S-05	마스터 카트는 IMU yaw 값과 Odometry 데이터를 이용하여 자신의 위치 및 방향 정보를 추정한다.
S-06	마스터 카트는 RealSense 카메라의 `/image_raw` 데이터를 YOLOv8 nano에 입력하여 사람 또는 장애물 객체 인식을 수행한다.
S-07	마스터 카트는 SLAM, Odometry, IMU를 기반으로 자신의 위치 좌표와 경로 정보를 생성한다.
S-08	마스터 카트는 생성된 위치 좌표, 방향, 경로 정보를 UDP 방식으로 슬레이브 카트에 실시간 송신한다.
S-09	슬레이브 카트는 UDP 수신 노드를 통해 마스터의 위치 데이터 및 경로 데이터를 수신한다.
S-10	슬레이브 카트는 마스터 위치 데이터와 자신의 IMU/Odometry 데이터를 비교하여 거리 오차와 방향 오차를 계산한다.
S-11	슬레이브 카트는 거리 오차 및 방향 오차를 기반으로 선속도와 각속도 명령을 생성한다.
S-12	슬레이브 카트는 생성된 제어 명령을 `/cmd_vel` 형태로 변환하여 MD400T 모터 드라이버에 전달한다.
S-13	슬레이브 카트는 Wi-Fi 통신이 일정 시간 이상 끊길 경우, 마스터 위치 데이터 기반 추종을 중단하고 USB Webcam을 통한 ArUco 마커 인식 및 이를 기반으로 한 보조 추종 모드로 자동 전환한다.

S-14	초음파 센서에서 50 cm 이내의 장애물이 감지될 경우, 슬레이브 카트는 추종 명령보다 정지 명령을 우선 적용한다.
S-15	시스템은 마스터와 슬레이브 간 통신 상태를 주기적으로 확인하고, 패킷 지연 또는 미수신 상태를 감지한다.
S-16	슬레이브는 마스터로부터 수신한 속도, 방향 명령을 적용하여 추종 제어를 수행해야 한다.
S-17	시스템은 비상 정지, 통신 끊김, 장애물 감지, 수동 정지 이벤트 발생 시 로그를 기록한다.

2.3 마스터 성능 요구사항

번호	요구 사항
P-01	슬레이브-마스터 간 목표거리 1m를 기준으로 추종이 작동하도록 한다.
P-02	마스터 명령을 10 Hz로 생성하고, 슬레이브 적용까지의 종단 지연을 200 ms 이내로 한다.
P-03	전방 0~5 m, 수평 시야각 86° 범위에서 사람·카트 크기 장애물을 90% 이상 탐지한다.
P-04	ROS2 환경에서 패킷 손실률 5% 이하, 재전송으로 인한 명령 공백을 0.5초 이내로 한다.
P-05	군중·기둥 등에 의한 일시 가림 후 목표를 2초 이내에 재획득한다.
P-06	정속 주행 최대 속도를 0.2 m/s로 한다.
P-07	실내 공항 유사 환경에서 연속 1시간 이상 운용한다.
P-08	TensorRT를 이용해 데이터를 경량화한다.
PR-09	사람을 1 m 이내로 감지하면 정지한다.
PR-10	SLAM 위치 추정 오차를 ±0.1 m 이내로 유지한다.

PR-11	지도 격자 크기를 0.05 m(5 cm) 해상도로 한다.
PR-12	장애물 회피 후 원래 경로로 자동 복귀한다.
PR-13	목표 지점 ±0.5 m 이내에 도달한다.

3 요구 기능 설계

3.1 하드웨어 기능 설계

3.1.1 H-F-01: 메인 제어 장치는 **ROS2 Humble** 환경에서 **LiDAR, RealSense** 카메라**, IMU** 데이터를 실시간 처리할 수 있어야 한다**.**

H-F-01	메인 제어 유닛(MCU)은 ROS 2 구동을 위한 연산 성능을 보장해야 한다.

• Ubuntu Linux 기반의 ROS 2 Humble 환경 구축

• NVIDIA Jetson Orin Nano를 마스터 제어 장치로 사용하여 LiDAR, RealSense 카메라, IMU 데이터 실시간 처리

3.1.2 H-F-02: 정밀 주행 제어를 위한 피드백 구동 시스템을 구축해야 한다**.**

H-F-02	정밀 주행 제어를 위한 피드백 구동 시스템을 구축해야 한다.

• 마스터: MDH300 모터 및 MD400T 모터 드라이버를 사용하여 자율주행 구동 수행

• 슬레이브: MDH200 모터 및 MD400T 모터 드라이버를 사용하여 추종 주행 수행

• 엔코더 피드백을 통해 정밀한 속도 및 위치 제어 구현

3.1.3 H-F-03: 주변 환경 인식 및 SLAM을 위한 2D LiDAR 센서를 장착해야 한다.

H-F-03	주변 환경 인식 및 SLAM을 위한 2D LiDAR 센서를 장착해야 한다.

• 차체 상단에 YDLIDAR G4를 장착하여 360도 환경 스캔

• LiDAR 스캔 데이터를 활용하여 Cartographer SLAM 기반 지도 생성 및 자율주행 위치 추정 수행

3.1.4 H-F-04: 안정적인 시스템 구동을 위한 전원 분배 시스템(PDU)을 갖춰야 한다.

H-F-04	안정적인 시스템 구동을 위한 전원 분배 시스템(PDU)을 갖춰야 한다.

• LV706-50E 리튬 배터리(7S, 충전 전압 29.4V)를 주 전원으로 사용

• 슬레이브: Buck Converter를 이용하여 제어기에 필요한 전압 공급

• 마스터: 전원 변환 모듈을 통해 Jetson Orin Nano 동작 전압에 최적화된 전원 공급

3.1.5 H-F-05: 화물 적재 및 주행 안정성을 고려한 차체(Chassis) 프레임을 설계해야 한다.

H-F-05	화물 적재 및 주행 안정성을 고려한 차체 프레임을 설계해야 한다.

• 기존 카트 프레임을 활용하여 자율주행 센서 및 제어 장치 탑재

• 차동 구동 방식(Differential Drive)을 적용하여 직진, 회전 및 제자리 회전 제어 용이성 확보

• 적재 하중을 고려한 견고한 구조 설계로 주행 안정성 유지

3.1.6 최종 하드웨어 결과물

[마스터]



[슬레이브]

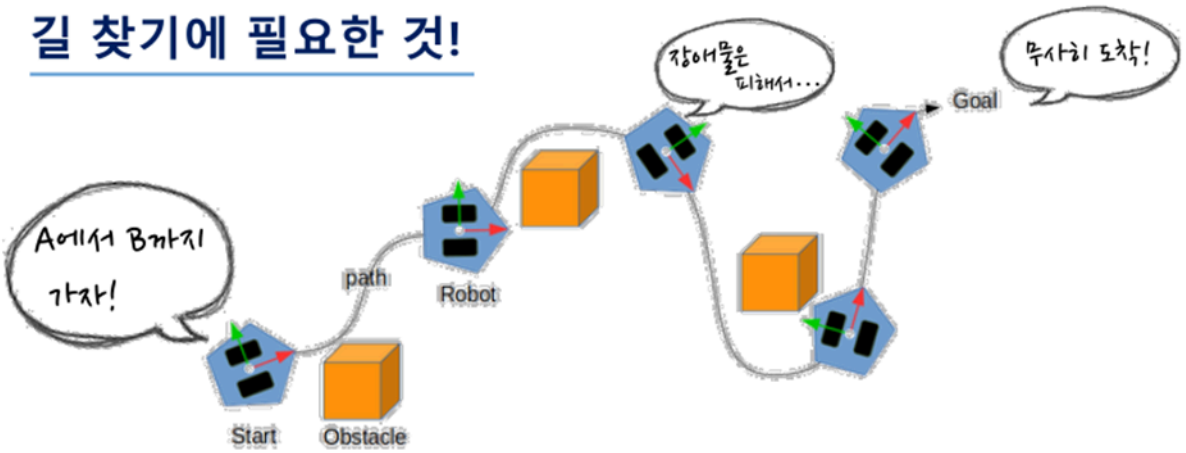


3.2 소프트웨어 기능 설계

3.2.1 S-M-01: 마스터 카트는 ROS2 Nav2를 사용하여 경로를 생성하고 자율주행을 수행해야 한다.

S-M-01	마스터 카트는 ROS2 Nav2를 사용하여 경로를 생성하고 자율주행을 수행해야 한다.

- Nav2의 SLAM(현 위치 지도 작성)기반 위치추정 기능 사용
- LiDAR/IMU/Odom 데이터로 지도 기반 경로 생성(10Hz 이상)
- Behavior Tree 기반 글로벌/로컬 플래너 적용
- 생성된 /cmd_vel 명령을 모터 드라이버로 전달



① **위치**: 로봇의 위치 계측/추정하는 기능
② **센싱**: 벽, 물체 등의 장애물의 계측하는 기능
③ **지도**: 길과 장애물 정보가 담긴 지도
④ **경로**: 목적지까지 최적 경로를 계산하고 주행하는 기능

3.2.2 S-M-02:마스터는 장애물 및 사람 감지 시 즉시 정지·회피를 수행해야 한다.

S-M-02	마스터는 장애물 및 사람 감지 시 즉시 정지·회피를 수행해야 한다.

• LiDAR 기반 전방 장애물 감지 및 카메라(YOLO) 기반 사람 감지

• 사람이 1m 이내 진입 시 정지 또는 회피 수행

• Nav2 costmap 연동으로 장애물 충돌 방지

3.2.3 S-M-03: 마스터-슬레이브 간 제어 및 상태 정보를 무선으로 양방향 통신해야 한다.

S-M-03	마스터-슬레이브 간 제어 및 상태 정보를 무선으로 양방향 통신해야 한다.

• 마스터→슬레이브: 속도, 경로, 상태 플래그, 장애물 이벤트

• 슬레이브→마스터: 통신상태, 장애물 이벤트

3.2.4 S-M-04: 자신의 현재 상태(위치, 속도, 상태 플래그)를 슬레이브 카트로 안정적이고 주기적(예: 20 Hz)으로 전송할 수 있어야 한다.

S-M-04	자신의 현재 상태(위치, 속도, 상태 플래그)를 슬레이브 카트로 안정적이고 주기적으로(예: 20 Hz) 전송할 수 있어야 한다.

•WiFi UDP로 위치·속도 정보 실시간 송신

•패킷 유실 시 최신 패킷을 우선 적용하여 지연 최소화

3.2.5 S-M-05: ROS2 네임스페이스 기반 다중 슬레이브 관리 기능을 제공해야 한다.

S-M-05	ROS2 네임스페이스 기반 다중 슬레이브 관리 기능을 제공해야 한다.

• /slaves/slave_1, /slaves/slave_2 구분 운영

• 마스터는 각 슬레이브를 고유 ID·네임스페이스로 제어

• 확장 시 다수 카트 군집 제어 가능

3.2.6 S-M-06: 마스터는 RViz를 통해 주행 상태를 시각화해야 한다.

S-M-06	마스터는 RViz를 통해 주행 상태를 시각화해야 한다.

•RViz로 지도,라이다 스캔, 로봇의 위치를 실시간으로 표시

•계획된 경로와 costmap을 화면으로 확인

•2D Goal Pose로 목적지 지정 가능

3.2.7 S-M-07:마스터는 좌표 반환을 통해 센서 데이터와 위치 정보를 통합해야 한다.

S-M-07	마스터는 좌표 반환을 통해 센서 데이터와 위치 정보를 통합해야 한다.

• map - odom - base_link - 센서 순의 좌표 변환 체계 구성

• 라이다 및 카메라 데이터를 로봇 위치 기준으로 정렬

• Cartographer가 map-odom, 휠 오도메트리가 odom-base_link 발행

3.2.8 S-M-08: 승인되지 않은 장치는 제어망에 접근할 수 없어야 한다.

S-M-08	승인되지 않은 장치는 제어망에 접근할 수 없어야 한다.

• 마스터 → 슬레이브: 속도, 경로, 상태 플래그

• Keystore 생성 및 노드 단위 인증서 발급

• ROS2로 네트워크 접근 제어

3.2.9 S-M-09: 마스터는 엔코더 오도메트리로 이동거리와 방향을 계산해야 한다.

S-M-09	마스터는 엔코더 오도메트리로 이동거리와 방향을 계산해야 한다.

• 좌우 바퀴 엔코더로 차동 구동 이동거리·회전각 계산

•TF(map→odom→base_link)로 센서 데이터와 위치 정보 통합

3.2.10 S-M-10:마스터는 GPU(CUDA)로 YOLO 객체 인식을 실시간 처리해야 한다.

S-M-10	마스터는 GPU(CUDA)로 YOLO 객체 인식을 실시간 처리해야 한다.

• Jetson GPU로 YOLO 추론 가속

• 실시간 사람 감지 성능 확보

3.2.11 S-S-01: 슬레이브는 마스터가 전송하는 속도/경로 명령을 수신하여 주행 제어를 수행해야 한다**.**

S-S-01	슬레이브는 마스터가 전송하는 속도/경로 명령을 수신하여 주행 제어를 수행해야 한다.

- /slaves/slave_1/cmd_vel 토픽으로 선속도·각속도 수신
- 수신한 명령을 모터 드라이버 PWM 신호로 변환
- 제어 주기 10Hz 이상 유지

3.2.12 S-S-02: 슬레이브는 마스터와의 추종 거리를 목표값**(**예**: 1m±**허용 오차**)**으로 유지해야 한다**.**

S-S-02	슬레이브는 마스터와의 추종 거리를 목표값(예: 1m±허용 오차)으로 유지해야 한다.

- 마스터 위치 데이터 및 카메라 ArUco 마커로 거리 측정
- 목표 거리와 실측 거리 오차를 P 제어로 보정
- 오차가 임계값 초과 시 감속 또는 정지

3.2.13 S-S-03: 슬레이브는 전방 장애물 감지 시 감속·정지하고 장애물 이벤트를 마스터에게 보고해야 한다**.**

S-S-03	슬레이브는 전방 장애물 감지 시 감속·정지하고 장애물 이벤트를 마스터에게 보고해야 한다.

- 초음파 센서 거리값이 임계거리(예: 0.5m) 이하이면 위험 상황으로 판단
- 현재 속도를 0으로 설정하고 BLOCKED 상태로 전환
- /slaves/slave_1/events 토픽에 장애물 감지 이벤트 발행

3.2.14 S-S-04: 슬레이브는 마스터와의 통신 타임아웃 발생 시 자동으로 안전 정지 모드로 전환해야 한다.

S-S-04	슬레이브는 마스터와의 통신 타임아웃 발생 시 자동으로 안전 정지 모드로 전환해야 한다.

- 일정 시간(예: 500ms) 동안 마스터 명령 미수신 시 타임아웃 판정
- 타임아웃 시 선속도·각속도를 0으로 설정하고 정지
- 상태 플래그를 LOST로 설정해 마스터에 보고

3.2.15 S-S-05: 슬레이브는 배터리·통신·동작 상태를 주기적으로 마스터에 전송해야 한다.

S-S-05	슬레이브는 배터리·통신·동작 상태를 주기적으로 마스터에 전송해야 한다.

- /slaves/slave_1/status 토픽으로 1~2Hz 주기로 상태 메시지 발행
- 포함 정보: 배터리 잔량, FOLLOWING/BLOCKED/LOST 등 동작 모드, 오류 코드
- 마스터 대시보드에서 슬레이브별 상태 모니터링

3.2.16 S-S-06: 슬레이브는 비상 정지 입력(물리 스위치 또는 **UI** 버튼 등)과 운용 모드를 소프트웨어 상태로 관리해야 한다.

S-S-06	슬레이브는 비상 정지 입력(물리 스위치 또는 UI 버튼 등)과 운용 모드를 소프트웨어 상태로 관리해야 한다.

• E-Stop 스위치 또는 UI 버튼 입력을 감지하면 즉시 모터 출력 차단

• 현재 모드(IDLE/FOLLOWING/BLOCKED/EMERGENCY)를 상태 메시지에 포함

• 마스터 UI에서 각 슬레이브 모드를 확인·수동 전환 가능

3.2.17 S-S-07: 슬레이브는 Wi-Fi 통신이 끊기면 카메라 ArUco 마커 추종으로 자동 전환해야 한다.

S-S-07	슬레이브는 Wi-Fi 통신이 끊기면 카메라 ArUco 마커 추종으로 자동 전환해야 한다.

•WiFi 위치 데이터 수신 중단 시 카메라 모드로 전환

• USB 웹캠으로 마스터의 ArUco 마커 인식

• 마커 거리·방향 기반 P 제어로 추종 지속

3.2.18 S-N-01: 주행 동기화를 위한 실시간 제어 데이터를 멀티캐스팅해야 한다.

S-N-01	주행 동기화를 위한 실시간 제어 데이터를 멀티캐스팅해야 한다.

• 통신 방식: **UDP (Best Effort)** 프로토콜 사용

• 마스터는 목표 속도(Linear)와 조향각(Angular)을 **10Hz 이상** 주기로 송신

• 패킷 유실 시 재전송을 기다리지 않고 **최신 패킷을 우선 적용**하여 지연 최소화

3.2.19 S-N-02: 마스터-슬레이브 간 양방향 상태 정보를 동기화해야 한다.

S-N-02	마스터-슬레이브 간 양방향 상태 정보를 동기화해야 한다.

• 통신 방식: UDP (Reliable) 사용

• 슬레이브는 배터리, 현재 속도, 에러 코드를 1Hz 주기로 마스터에 보고

• 마스터는 수집된 전체 시스템 상태(Global Status)를 네트워크에 브로드캐스팅

3.2.20 S-N-03: 통신 연결 상태 감지 및 타임아웃 처리를 수행해야 한다.

S-N-03	통신 연결 상태 감지 및 타임아웃 처리를 수행해야 한다.

• 통신 방식: UDP 기반 연결 확인

• 500ms 주기로 Heartbeat 신호를 교환하여 연결 유효성 검증

• 임계 시간(예: 5.0초) 초과 시 통신 두절(Lost)로 판단하고 즉시 멈춤(stop)

3.2.21 S-N-04: 긴급 상황 발생 시 최우선순위로 정지 명령을 전파해야 한다.

S-N-04	긴급 상황 발생 시 최우선순위로 정지 명령을 전파해야 한다.

• 통신 방식: UDP사용,정지 명령을 반복 전송하여 전달 보장

• 장애물 감지 또는 지오펜싱 이탈 시 ' Stop' 플래그 생성

• 주행 제어 패킷보다 높은 우선순위로 즉시 전송하여 슬레이브 제동 수행

3.2.22 S-N-05: ROS2 네임스페이스 기반 다중 슬레이브 식별 체계를 갖춰야 한다.

S-N-05	ROS2 네임스페이스 기반 다중 슬레이브 식별 체계를 갖춰야 한다.

• 각 슬레이브를 고유 네임스페이스(예: slaves_cart_1, slaves_cart_2)로 구분

• 마스터는 ROS 2 Discovery 기능을 통해 동적으로 접속된 슬레이브 식별

• 단일 마스터가 N대의 슬레이브를 개별 또는 군집 제어 가능하도록 설계

3.2.23 S-N-06: 마스터와 슬레이브가 멀어졌을 시 마스터는 대기해야 한다.

S-N-06	마스터와 슬레이브가 멀어졌을 시 마스터는 대기해야 한다.

•슬레이브와의 거리가 임계값을 초과하면 마스터는 주행을 멈추고 대기

•슬레이브가 추종 거리 내로 복귀하면 자동으로 주행 재개

•대기 상태를 UI 대시보드에 표시

3.2.24 S-N-07: Wi-Fi 기반 제어 네트워크 인프라를 구축해야 한다.

S-N-07	Wi-Fi 기반 제어 네트워크 인프라를 구축해야 한다.

•마스터 카트(또는 라우터)는 독립적인 SSID를 생성하여 폐쇄망 형성

•WPA2 보안 키 또는 MAC 주소 필터링을 통한 비인가 기기 접속 차단

•DHCP를 통해 각 노드(마스터/슬레이브/UI)에 고정 IP 대역 할당 및 관리

3.3 성능 요구사항 설계

3.3.1 성능 요구사항 개요

. 본 시스템의 성능 요구사항은 공항 및 대형 실내 공간에서 안정적인 수하물 운반, 마스터-슬레이브 추종, 장애물 대응, 통신 안정성, 저비용 운용 가능성을 검증하기 위한 기준으로 설정한다.

3.3.2 PR-01 최소 적재 하중 보장

PR-01	최소 적재하중(Payload) 및 견인력을 보장해야 한다.

기본 하중: 최소 50 kg 이상의 화물을 적재한 상태에서 정상 주행이 가능해야 한다.

검증 방법: 50kg 이상의 화물을 적재하고 직진, 회전, 정지 주행 테스트를 수행한다.

판정 기준: 주행 중 모터 정지, 바퀴 헛도는 정도, 구조물 흔들림이 발생하지 않아야 한다.

3.3.2 PR-02: 목표 적재 하중 보장

PR-02	여행용 캐리어 수준의 화물 운반을 고려하여 목표 적재 하중을 보장해야 한다.

목표 하중: 최대 20 kg까지 견인 가능하도록 설계한다.

검증 방법: 단계별 하중 증가 후 직선 주행 및 회전 주행 테스트를 수행한다.

판정 기준:표 하중 조건에서 정상 출발, 정지, 회전이 가능해야 한다.

3.3.3 PR-03: 안전 추종 속도 유지

PR-03	안전한 추종 주행을 위한 적정 주행 속도를 유지해야 한다.

평상시 주행 속도: 1.0~1.2 m/s를 유지한다.

최대 주행 속도: 최대 1.5 m/s까지 가속 가능해야 한다.

최저 주행 속도: 0.1 m/s 단위의 저속 제어가 가능해야 한다.

판정 기준: 보행자 추종 중 급가속, 급정지 없이 안정적으로 주행해야 한다.

3.3.4 PR-04: 마스터-슬레이브 통신 성능

PR-04	마스터와 슬레이브 간 위치 데이터 송수신 성능을 확보해야 한다.

통신 방식: Wi-Fi AP 기반 UDP 통신을 사용한다.

송신 데이터: 마스터 위치, 방향, 경로, 상태 플래그를 포함한다.

목표 주기: 최소 10 Hz 이상의 송수신을 수행한다.

이상 판단: 500ms 이상 데이터 미수신 시 통신 이상으로 판단한다.

3.3.5 PR-05: 장애물 감지 및 정지 성능

PR-05	슬레이브 카트의 근거리 장애물 감지 및 정지 성능을 확보해야 한다.

센서 구성: 초음파 센서 4개를 사용한다.

정지 조건: 장애물 거리 50 cm 이내 감지 시 정지한다.

우선순위: 장애물 정지 명령은 일반 추종 명령보다 우선한다.

판정 기준: 장애물 감지 후 모터 구동이 즉시 중단되어야 한다.

3.3.6 PR-06: ArUco 보조 추종 성능

PR-06	통신 불안정 상황에서 카메라 기반 보조 추종 성능을 확보해야 한다.

센서 구성: USB Webcam을 사용한다.

인식 대상: 마스터 후방 또는 지정 위치의 ArUco 마커를 인식한다.

사용 조건: Wi-Fi 통신 끊김 또는 위치 데이터 불안정 발생 시 사용한다.

판정 기준: 마커 인식 시 마스터와의 상대 거리 및 방향 계산이 가능해야 한다.

3.3.7 PR-07: 방향 보정 성능

PR-07	슬레이브 카트의 방향 오차 보정 성능을 확보해야 한다.

센서 구성: EBIMU yaw 데이터를 사용한다.

보정 방식: 마스터 경로 방향과 슬레이브의 현재 방향을 비교하여 회전 명령을 생성한다.

판정 기준: 추종 주행 중 진행 방향이 급격히 틀어지지 않아야 한다.

3.3.8 PR-08: 자율주행 경로 생성 기능

PR-08	마스터 카트의 실내 자율주행 경로 생성 성능을 확보해야 한다.

지도 생성: 2D LiDAR 기반 Cartographer SLAM을 사용하여 실내 지도를 생성한다.

경로 생성: Map 기반으로 Nav2 기반 목적지 경로 생성을 수행한다.

출력 명령: Nav2에서 생성된 경로를 기반으로 `/cmd_vel` 모터 제어 명령을 생성한다.

판정 기준: 지정 목적지까지 경로를 생성하고 충돌 없이 주행이 가능해야 한다.

3.3.9 PR-09: 객체 인식 성능

PR-09	마스터 카트의 사람 및 장애물 객체 인식 성능을 확보해야 한다.

센서 구성: RealSense D455를 사용한다.

인식 모델: YOLOv8 nano를 사용한다.

활용 목적: 사람 또는 장애물 감지 후 회피 또는 정지 판단에 활용한다.

판정 기준: 주행 경로상 주요 장애물 인식이 가능해야 한다.

3.3.10 PR-10: 안전 정지 기능

PR-10	비상 상황 발생 시 시스템의 안전 정지 성능을 확보해야 한다.

정지 조건: 비상 정지 입력, 장애물 감지, 통신 끊김, 제어 이상 발생 시 정지한다.

정지 방식: 모터 드라이버에 정지 명령을 전달한다.

판정 기준: 안전 조건 발생 시 일반 주행 명령보다 정지 명령을 우선 적용해야 한다.

3.3.11 PR-11: 다중 슬레이브 확장 성능

PR-11	마스터 1대에 여러 슬레이브 카트를 연결할 수 있는 확장 성능을 확보해야 한다.

관리 방식: 슬레이브별 고유 ID를 기반으로 관리한다.

통신 방식: 각 슬레이브에 대한 위치 데이터 및 상태 데이터를 구분한다.

활용 목적: 군집 주행 및 대규모 수하물 운반량 확장에 활용한다.

판정 기준: 슬레이브 추가 시 기본 통신 및 상태 관리가 가능해야 한다.

3.3.12 PR-12: 전원 안전성 성능

PR-12	주행 중 전원 불안정으로 인한 시스템 정지를 방지해야 한다.

슬레이브 제어부 전원: Raspberry Pi 4에 5V를 공급한다.

슬레이브 구동부 전원: MD400T 및 모터에 24V를 공급한다.

검증 방법: 주행 중 제어부 재부팅, 통신 끊김, 모터 출력 저하 여부를 확인한다.

판정 기준: 정상 주행 중 전원 불안정으로 인한 시스템 종료가 없어야 한다.

4 테스트

4.1 테스트 주의사항 및 기타 고려사항

- 사람 인식 기반 정지 기능을 테스트하기 전에는 박스와 같은 더미 장애물을 이용하여 기본 정지 동작을 먼저 검증하였다.
- 테스트 전에는 배터리 잔량, 전원 연결 상태, 바퀴 고정 상태, 센서 연결 상태를 점검하여 주행 중 발생할 수 있는 오류를 최소화하였다.
- 기능 검증은 마스터 단독 주행, 슬레이브 단독 구동, 마스터-슬레이브 연동 주행 순서로 단계적으로 진행하였다.
- 전체 시스템에서는 사람이 감지되면 마스터가 정지하고, 슬레이브는 마스터의 정지 상태를 수신하여 함께 정지하는 방식으로 안전 동작을 구현하였다.

4.1.1 테스트 케이스

4.1.2 기능 단위 테스트

번호	테스트 케이스	테스트 결과	요구사항 번호
TC-001	• **조건:** 마스터가 슬레이브에게 속도 및 방향 명령을 전송함 • **예상:** 슬레이브가 수신된 명령과 동일한 속도 및 방향으로 구동됨 • **요구사항:** S-S-01	O	*S-S-01*
TC-002	• **조건:** 주행 경로상 장애물이 없는 환경	O	*S-S-02*

	• **예상:** 슬레이브가 마스터와 1m ± 0.2m 간격을 유지하며 추종함 • **요구사항:** S-S-02		
TC-003	• **조건:** 슬레이브 전방 임계거리 이내 장애물(사람, 박스 등) 진입 • **예상:** 즉시 감속 및 정지 후 장애물 이벤트를 마스터에 보고함 • **요구사항:** S-S-03	O	*S-S-03*
TC-004	• **[테스트 절차]** • 마스터/슬레이브 전원 On 및 FOLLOWING 모드 직선 주행 시작 • 주행 중 물리 비상 정지 스위치 작동 • **[결과 확인]** • 슬레이브 즉시 정지 및 대시보드 상 EMERGENCY 모드 전환 확인 • **요구사항:** S-S-06	O	*S-S-06*
TC-005	• **조건:** 마스터-슬레이브 무선 통신 연결 상태 • **예상:** 슬레이브 배터리 및 동작 모드 정보가 마스터 UI에 실시간 갱신됨 • **요구사항:** S-S-05	O	*S-S-05*
TC-006	• **[테스트 절차]** • FOLLOWING 모드 주행 중 Wi-Fi 연결 강제 차단	O	*S-S-07*

	• **[결과 확인]** • 카메라 기반 ArUco 마커 인식 및 보조 추종 모드로의 자동 전환 확인 • **요구사항:** S-S-07		
TC-007	• **[테스트 절차]** • LiDAR 스캔 및 Cartographer SLAM 실행 • YOLOv8n GPU 가속 기반 객체 인식 및 Depth 측정 활성화 • **[결과 확인]** • 안정적인 지도 작성 및 1m 이내 사람 감지 시 정지 동작 확인 • **요구사항:** S-M-10	O	*S-M-10*

4.1.2.1 마스터/슬레이브 개별 기능 검증

번호	테스트 케이스	테스트 결과
TC-008	**슬레이브 카메라의 ArUco 마커 인식 확인** (조건: ID 0번 마커 전방 배치 / 예상: ID, 좌표, 거리 출력 / 요구사항: S-FR-01)	O
TC-009	**마커 멀어짐에 따른 전진 추종 확인** (조건: 마커를 목표 거리보다 멀리 이동 / 예상: /cmd_vel의 linear.x 발생 / 요구사항: S-FR-02)	O

TC-010	**좌우 이동 시 방향 보정 확인** (조건: 마커를 좌우 이동 / 예상: /cmd_vel의 angular.z 발생 / 요구사항: S-FR-03)	O
TC-011	**목표 거리 이내 접근 시 정지 확인** (조건: 마커를 슬레이브 가까이 이동 / 예상: 선속도 0에 근접 / 요구사항: S-FR-04)	O
TC-012	**마커 미인식 시 정지 확인** (조건: 마커 가림 또는 화면 밖 이동 / 예상: /cmd_vel 0으로 변경 / 요구사항: S-FR-05)	O
TC-013	**비상 정지 버튼 입력 시 정지 확인** (조건: 주행 중 비상 정지 버튼 입력 / 예상: 모터 즉시 정지 / 요구사항: S-FR-06)	O

4.1.3 시스템 통합 테스트

번호	테스트 케이스	테스트 결과
TC-014	• **조건:** 마스터가 Wi-Fi를 통해 경로 및 위치(x, y, yaw) 데이터 송신 • **예상:** 슬레이브가 UDP 패킷을 정상 수신하여 ROS2 노드에서 데이터 확인 가능 • **요구사항:** S-09, S-10	O
TC-015	• **조건:** 슬레이브를 직선 방향으로 일정 거리 이동 • **예상:** 모터 인코더 기반 Odometry 값이 실제 이동 방향/거리와 일치하여 증가	O

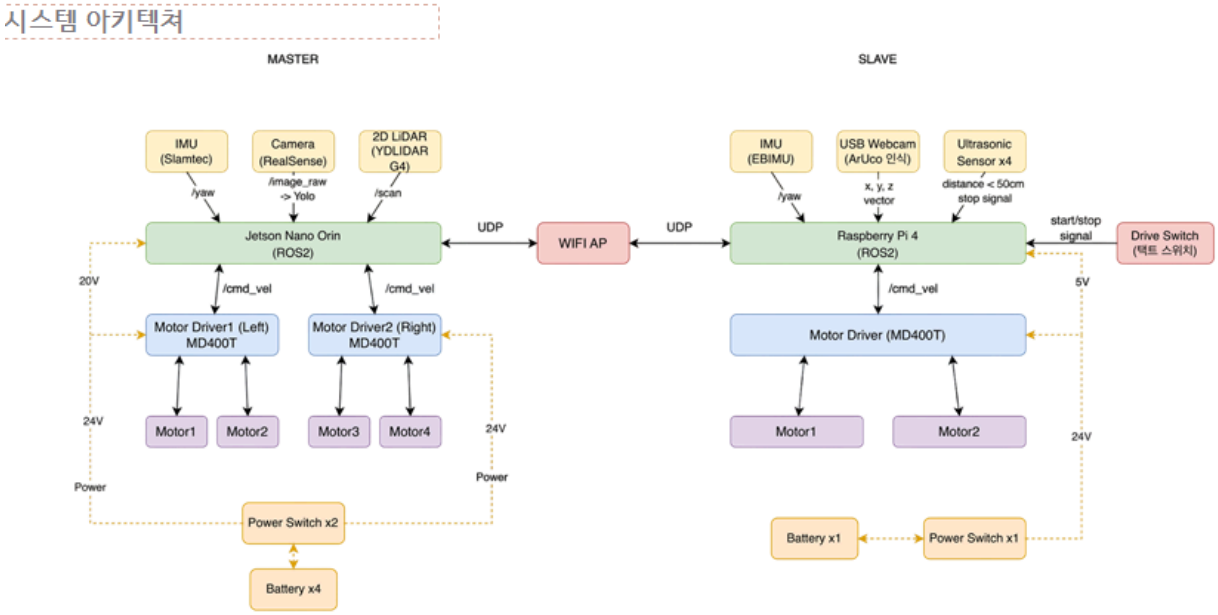
	• **요구사항:** H-S-03, S-10	
TC-016	• **조건:** 슬레이브 카트를 좌우로 회전 시킴 • **예상:** EBIMU yaw 값이 회전량에 따라 변화하며 ROS2 토픽에 정상 반영됨 • **요구사항:** H-S-04, PR-12	O
TC-017	• **조건:** 슬레이브가 전진 주행 중 좌우로 방향이 틀어짐 • **예상:** IMU 기반 방향 오차 계산을 통해 angular.z 보정 명령이 생성됨 • **요구사항:** S-10, S-11	O
TC-018	• **조건:** 마스터가 생성된 직선 경로 데이터를 송신 • **예상:** 슬레이브가 경로 방향으로 전진하며 linear.x 속도 명령이 발생함 • **요구사항:** S-11, S-12	O
TC-019	• **조건:** 슬레이브의 헤딩을 마스터 경로 방향과 다르게 초기 배치 • **예상:** yaw 오차를 즉각 계산하여 경로 정렬을 위한 좌/우 회전 명령 생성 • **요구사항:** H-S-14, PR-12	O
TC-020	• **조건:** 마스터와 슬레이브 간 거리를 목표값보다 멀게 배치 • **예상:** 거리 오차에 비례하여 가속하고, 목표 거리 도달 시 감속/정지함 • **요구사항:** H-S-14, PR-03	O
TC-021	• **조건:** 슬레이브가 마스터 경로의 횡방향(좌/우)에서 이탈함 • **예상:** 횡방향 오차 보정 알고리즘을 통해 원래 경로로 재진입함 • **요구사항:** S-10, S-11	O
TC-022	• **조건:** 슬레이브 전방 50cm 이내에 물리적 장애물 배치	O

	• **예상:** 초음파 센서 감지로 추종 명령보다 정지 명령이 우선 적용됨 • **요구사항:** H-S-08, S-15	

4.2 테스트 케이스

번호	테스트 케이스	테스트 결과
TC-SYS-001	마스터, 슬레이브를 직선 복도 환경에서 출발점 → 목적지까지 연속 운행 시, 충돌 없이 슬레이브가 끝까지 추종하는지 확인	O
TC-SYS-002	주행 시, 임의로 사람을 앞에 세우고, 마스터가 즉시 정지 하는지 확인	O
TC-STS-003	TC-SYS-002를 이어, 사람이 없어졌을 경우 마스터가 목표 지점으로 다시 재주행하는 지 확인	O
TC-SYS-003	보행자 여러 명이 임의로 앞을 가로질러도 마스터가 정지하며, 슬레이브도 안전거리 50cm를 유지하는지 확인	O
TC-SYS-004	슬레이브와 하드웨어가 운용될 때, 각각 다른 ID/네임스페이스로 제어되며 충돌이나 통신 혼선 없이 목적지까지 도착하는지 확인	O

[전체 시스템 아키텍처]



본 시스템은 **마스터-슬레이브 기반 무선 추종 카트 구조**로 구성한다.

마스터 카트는 자율주행, 지도 생성, 객체 인식, 위치 정보 생성 및 송신을 담당하고 슬레이브 카트는 마스터로부터 수신한 위치 및 경로 데이터를 기반으로 추종 주행을 수행한다. 두 카트는 Wi-Fi AP를 중심으로 UDP 통신을 수행하며, 마스터의 위치 정보와 주행 상태가 슬레이브로 실시간 전달되는 구조를 가진다.

마스터 카트는 **Jetson Orin Nano**를 메인 제어 유닛으로 사용한다. Jetson Orin Nano는 ROS2 환경에서 LiDAR, 카메라, IMU 데이터를 수집하고 SLAM 기반 지도 생성, Nav2 기반 자율주행, YOLO 기반 객체 인식을 수행한다. 마스터는 시스템 전체에서 선행 차량 역할을 하며, 사용자가 지정한 목적지까지 먼저 이동하면서 슬레이브가 따라갈 기준 경로를 생성한다.

세부적으로 **마스터**에는 **2D LiDAR, RealSense 카메라, IMU 센서**가 연결된다. 2D LiDAR는 주변 환경을 스캔하여 `/scan` 데이터를 생성하고 이 데이터는 Cartographer SLAM으로 전달되어 실내 2D 지도를 작성하는 데 사용된다. 작성된 지도는 Occupancy Grid Map 형태로 구성되며, Nav2가 목적지까지의 경로를 계획하는 기준으로 사용된다. RealSense 카메라는 `/image_raw` 영상 데이터를 제공하고 YOLO 기반 객체 인식 알고리즘을 통해 사람 또는 장애물을 인식하는 데 사용된다. IMU는 마스터의 yaw 방향 정보를 제공하며 주행 중 발생하는 방향 오차를 보정하고 Odometry 및 Path 생성 과정에 활용된다.

마스터의 주행 제어는 Nav2에서 생성된 `/cmd_vel` 명령을 기반으로 수행된다. `/cmd_vel` 명령에는 선속도와 각속도 정보가 포함되며 해당 명령은 MD400T 모터 드라이버로 전달된다. 마스터 카트는 좌측 모터 드라이버와 우측 모터 드라이버를 각각 사용하고 있으며 총 4개의 모터를 제어한다. 이를 통해 마스터는 직진, 회전, 정지, 목적지 추종 주행을 수행한다.

마스터는 주행 중 자신의 위치, 방향, 경로 정보를 생성한다. 생성된 정보는 `x`, `y`, `yaw`, Path Point, 상태 플래그 등의 형태로 구성되며 Wi-Fi AP를 통해 UDP 방식으로 슬레이브 카트에 전송된다. 이때 UDP 통신을 사용하는 이유는 마스터와 슬레이브 간 위치 데이터를 빠르게 전달하여 실시간 추종 주행이 가능하도록 하기 위함이다. 따라서 마스터는 단순히 목적지까지 이동하는 카트가 아니라 슬레이브의 추종 기준을 생성하고 전달하는 중심 제어 장치 역할을 한다.

슬레이브 카트는 **Raspberry Pi 4**를 메인 제어 유닛으로 사용한다. 슬레이브는 마스터처럼 고성능 자율주행 기능을 모두 수행하지 않고 마스터로부터 전달받은 위치 및 경로 데이터를 기반으로 추종 주행을 수행한다. 이를 통해 슬레이브의 하드웨어 구성을 단순화할 수 있으며 전체 시스템의 제작 비용을 줄일 수 있다.

세부적으로 **슬레이브**에는 **EBIMU, USB Webcam, 초음파 센서 4개, MD400T 모터 드라이버, 물리적인 Drive Switch**가 연결된다. EBIMU는 슬레이브의 yaw 방향 정보를 측정하며, 마스터 경로를 추종할 때 슬레이브의 방향 오차를 보정하는 데 사용된다. USB Webcam은 마스터에 부착된 ArUco 마커를 인식하기 위한 센서로 사용된다. 마커가 인식되면 슬레이브는 마커의 x, y, z 벡터 정보를 계산하여 마스터와의 상대 거리와 방향을 추정한다. 이 기능은 Wi-Fi 통신이 불안정하거나 마스터의 위치 데이터 수신이 원활하지 않을 때 보조 추종 방식으로 활용된다.

초음파 센서 4개는 슬레이브 주변의 근거리 장애물을 감지한다. 초음파 센서는 장애물과의 거리를 측정하며 장애물이 50cm 이내에 위치할 경우 정지 신호를 생성한다. 이 정지 신호는 일반적인 추종 주행 명령보다 높은 우선순위를 가진다. 따라서 슬레이브가 마스터를 정상적으로 추종하는 중이더라도 전방 또는 주변에 장애물이 감지되면 즉시 모터 구동을 중단하도록 구성한다.

슬레이브의 주행 제어는 Raspberry Pi 4에서 생성한 `/cmd_vel` 명령을 MD400T 모터 드라이버로 전달하는 방식으로 수행된다. 슬레이브는 마스터로부터 수신한 위치 데이터와 자신의 IMU/Odometry 데이터를 비교하여 거리 오차와 방향

오차를 계산한다. 거리 오차가 크면 전진 속도를 조절하고 방향 오차가 발생하면 각속도 명령을 생성하여 좌우 방향을 보정한다. 최종적으로 MD400T 모터 드라이버는 좌우 모터 2개를 제어하여 슬레이브 카트가 마스터의 경로를 따라 이동하도록 한다.

Drive Switch는 슬레이브의 수동 주행 제어 및 비상 정지를 위한 입력 장치로 사용된다. 사용자가 Drive Switch를 통해 start 또는 stop 신호를 입력하면 슬레이브는 해당 신호를 우선적으로 처리한다. 특히 stop 신호가 입력될 경우 현재 추종 모드, ArUco 추종 모드, Odometry 기반 주행 상태와 관계없이 모터를 즉시 정지한다. 이를 통해 실험 환경 또는 실제 운용 환경에서 사용자가 직접 시스템을 제어할 수 있도록 한다.

전원 구조는 마스터와 슬레이브가 각각 독립적으로 구성된다. 마스터는 배터리 4개와 Power Switch 2개를 기반으로 전원을 공급받으며 Jetson Orin Nano와 센서부에는 제어용 전원을 공급하고, 모터 드라이버와 모터에는 구동용 24V 전원을 공급한다. 슬레이브는 배터리 1개와 Power Switch 1개를 기반으로 구성되며, Raspberry Pi 4 및 센서부에는 5V 전원을 공급하고 MD400T 모터 드라이버와 모터에는 24V 전원을 공급한다. 이처럼 제어부와 구동부의 전원을 분리하여 각 장치에 필요한 전압을 안정적으로 공급하도록 설계한다.

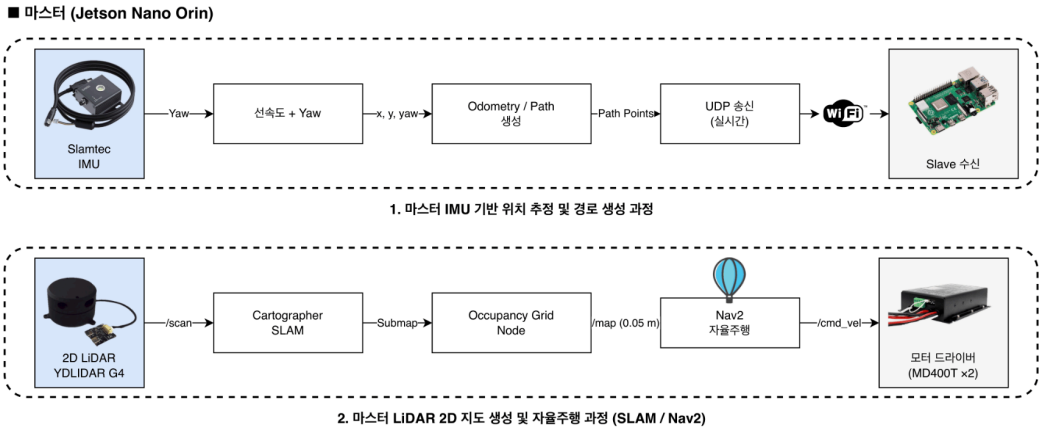
전체 동작 흐름은 다음과 같다.

먼저 마스터는 LiDAR와 IMU를 이용하여 자신의 위치를 추정하고, Cartographer

SLAM을 통해 실내 지도를 생성한다. 이후 사용자가 목적지를 지정하면 Nav2가 목적지까지의 경로를 계획하고, 마스터는 `/cmd_vel` 명령을 통해 자율주행을 수행한다. 동시에 마스터는 자신의 위치, 방향, 경로 데이터를 UDP 방식으로 슬레이브에 전송한다. 슬레이브는 수신한 데이터를 기반으로 자신의 IMU/Odometry 정보와 비교하여 추종 주행 명령을 생성한다. 통신이 불안정한 경우에는 USB Webcam을 이용해 ArUco 마커를 인식하고 마커 기반 보조 추종을 수행한다. 주행 중 초음파 센서가 장애물을 감지하면 슬레이브는 추종 명령보다 정지 명령을 우선 적용하여 안전 정지를 수행한다.

따라서 본 시스템은 **마스터의 고성능 자율주행 기능과 슬레이브의 저비용 추종 기능을 분리한 구조**라고 할 수 있다. 마스터는 고성능 센서와 연산 장치를 기반으로 경로 생성 및 자율주행을 담당하고, 슬레이브는 상대적으로 단순한 센서와 제어 구조를 통해 마스터를 추종한다. 이러한 구조는 모든 카트에 고가의 자율주행 장비를 탑재하지 않아도 되므로 제작 비용을 절감할 수 있으며, 필요에 따라 슬레이브 카트를 추가하여 수하물 운반량을 확장할 수 있다는 장점을 가진다.

[마스터 시스템 아키텍쳐]



본 시스템의 마스터는 Jetson Orin Nano를 메인 제어 유닛으로 사용하며, ROS2 환경에서 자율주행과 사람 인식을 수행한다.

마스터에는 세 가지 센서가 연결된다. 2D 라이다(YDLIDAR G4)는 360도로 주변을 스캔하여 /scan 데이터를 생성하고, RealSense 카메라는 /image_raw 영상을 제공하며, IMU(Slamtec)는 /yaw 방향 정보를 제공한다.

라이다의 /scan 데이터는 Cartographer로 전달되어 주변 환경 지도(map)를 작성하고 로봇의 위치를 추정한다. 작성된 지도를 기반으로 Nav2가 목적지까지의 경로를 계획하고 추종하여 /cmd_vel 속도 명령을 생성한다. 이 명령은 좌·우 두 개의 MD400T 모터 드라이버로 전달되며, 각 드라이버는 모터 2개씩 총 4개의 인휠 모터를 차동 구동 방식으로 제어한다.

동시에 RealSense 카메라의 영상은 YOLO로 전달되어 사람을 인식하고, 깊이(depth) 데이터로 거리를 측정한다. 사람이 1m 이내로 접근하면 마스터는 정지하거나 회피한다.

마스터는 자신의 위치 및 방향 정보를 WiFi AP를 거쳐 UDP 통신으로 슬레이브에 실시간 전송하여 추종 주행이 가능하도록 한다. 전원은 배터리 4개와 전원 스위치 2개로 구성되어 모터용 24V와 제어기용 전압을 안정적으로 공급한다.

-1. 마스터 IMU 기반 위치 추정 및 경로 생성 과정

마스터 1번 테스트는 IMU yaw 값과 Odometry 데이터를 이용하여 마스터의 위치 및 방향을 추정하고, 해당 정보를 슬레이브에 UDP 방식으로 송신하는지를 검증하는 테스트이다.

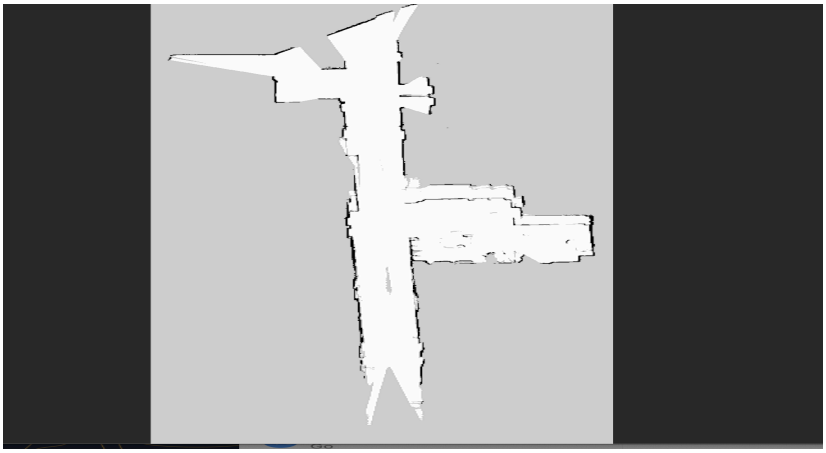
먼저 마스터 카트의 IMU 센서가 정상적으로 yaw 값을 출력하는지 확인한다. yaw 값은 마스터의 회전 방향을 나타내는 핵심 데이터이므로 마스터를 좌우로 회전시켰을 때 값이 변화해야 한다. yaw 값이 정상적으로 수신되지 않으면 마스터의 진행 방향을 정확히 계산할 수 없고 이후 생성되는 Path Point도 신뢰하기 어렵다.

다음으로 마스터는 선속도와 yaw 값을 기반으로 자신의 위치 변화를 계산한다. 직진 주행 시에는 x 또는 y 좌표가 일정한 방향으로 증가해야 하며 회전 주행 시에는 yaw 값이 함께 변화해야 한다. 이 데이터를 이용하여 Odometry가 생성된다.

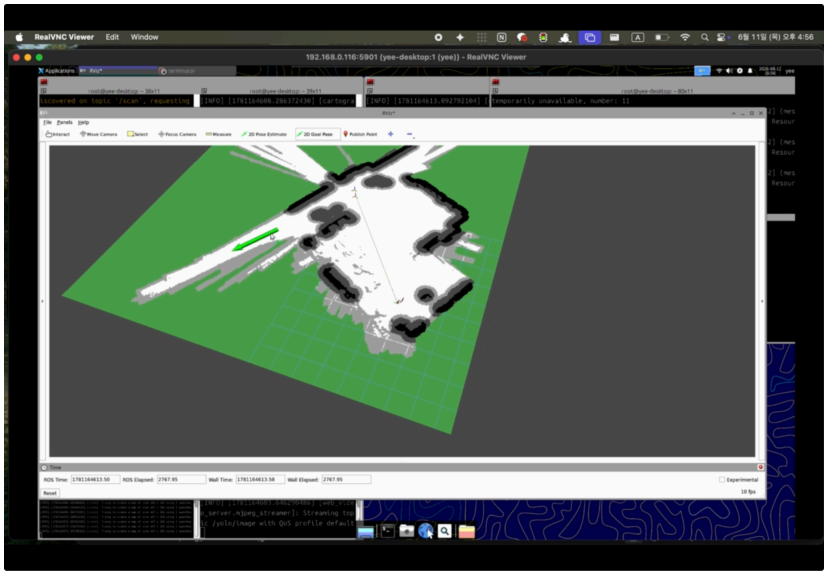
Odometry가 생성되면 마스터는 이동 경로를 Path Point 형태로 변환한다. Path Point는 슬레이브가 따라갈 기준 경로로 사용되므로, 마스터의 실제 이동 경로와 유사하게 생성되어야 한다. 이후 마스터는 생성된 x, y, yaw, path point 데이터를 Wi-Fi AP를 통해 UDP 방식으로 슬레이브에 송신한다.

이 테스트의 핵심은 마스터가 자신의 이동 정보를 단순히 내부적으로만 사용하는 것이 아니라, 슬레이브가 추종할 수 있는 형태의 경로 데이터로 변환하고 실시간으로 전송하는지 확인하는 것이다.

-2. 마스터 **LIDAR 2D** 지도 생성 및 자율 주행 과정

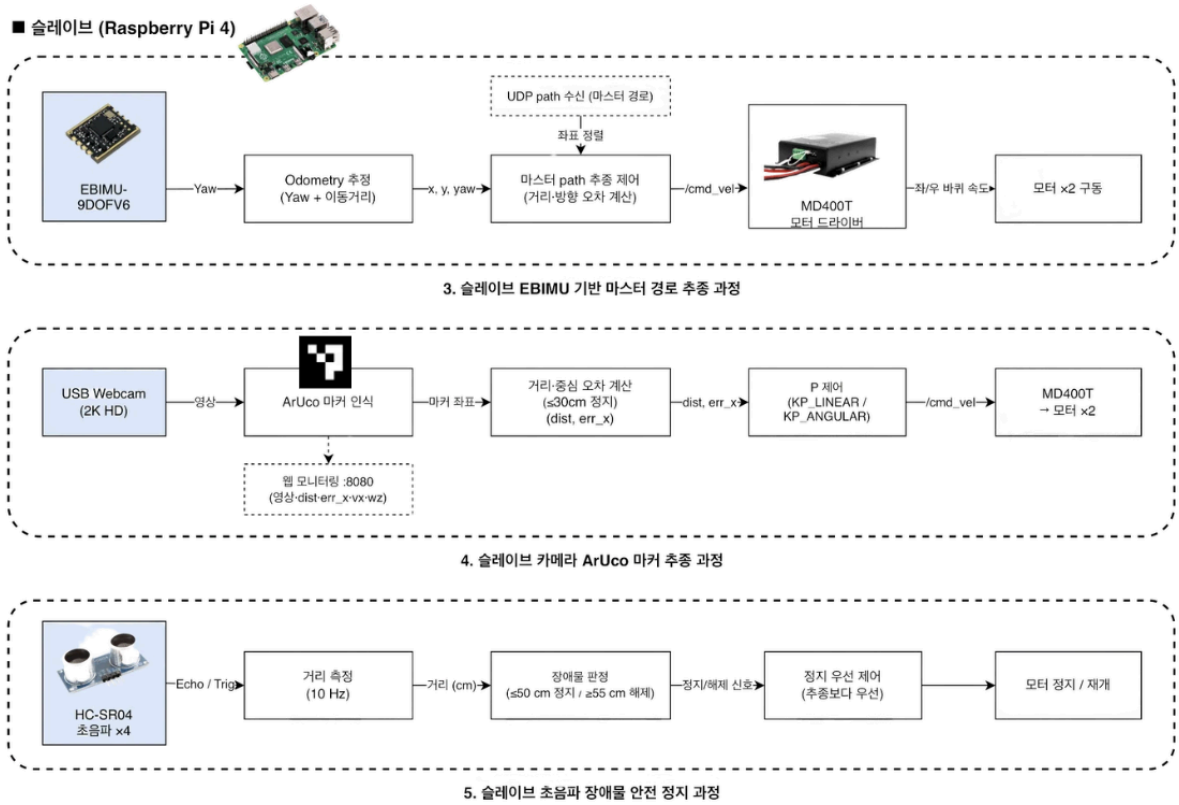


Cartographer로 복도를 주행하며 완성한 2D 격자 지도이다. 0.05 m(5 cm) 해상도로 작성되었으며 흰색 영역을 따라 Nav2가 목적지까지 경로를 계획한다.



Cartographer SLAM 실행 화면이다. 흰색은 이동 가능한 공간, 검은색은 벽·장애물, 회색은 미탐색 영역을 나타낸다. 초록색 화살표는 지정한 목적지(2D Goal Pose)이며 라이다(/scan) 데이터를 기반으로 실시간으로 지도를 작성하면서 로봇의 위치를 추정한다.

[슬레이브 시스템 아키텍처]



본 시스템의 슬레이브 카트는 Raspberry Pi 4를 메인 제어 유닛으로 사용하며 ROS2 환경에서 마스터 카트의 경로 데이터를 수신하고 추종 주행을 수행한다. 마스터와 슬레이브는 Wi-Fi AP를 통해 UDP 방식으로 데이터를 송수신하고 슬레이브는 수신한 마스터의 위치 및 경로 정보를 기반으로 자신의 이동 방향과 속도를 결정한다.

슬레이브 카트에는 EBIMU, USB Webcam, 초음파 센서 4개, MD400T 모터 드라이버, Drive Switch가 연결된다. EBIMU는 슬레이브의 yaw 방향 정보를 제공하며 Odometry 계산 과정에서 이동 방향 보정에 사용된다. USB Webcam은 ArUco 마커를 인식하여 마스터와의 상대 위치를 계산하는 보조 추종 센서로 사용된다. 초음파 센서 4개는 슬레이브 주변의 근거리 장애물을 감지하며 장애물이 50cm 이내에 위치할 경우 정지 신호를 생성한다.

Raspberry Pi 4에서 생성한 `/cmd_vel` 명령은 MD400T 모터 드라이버로 전달되며, MD400T는 좌우 모터 2개를 제어하여 슬레이브 카트의 전진, 정지, 회전 동작을 수행한다. 또한 물리적 switch는 비상 상황 또는 수동 정지 상황에서 모터 구동을 즉시 중단하는

역할을 한다.

전원 구조는 제어부와 구동부로 구분된다. Raspberry Pi 4 및 센서부에는 5V 전원이 공급되고 MD400T 모터 드라이버와 모터에는 24V 전원이 공급된다. 이를 통해 제어부와 구동부가 각각 요구하는 전압을 안정적으로 제공하며 주행 중 전원 불안정으로 인한 시스템 종료를 방지한다.

-3. 슬레이브 **EBIMU** 기반 마스터 경로 추종 과정

슬레이브는 마스터로부터 UDP 방식으로 경로 데이터를 수신한 뒤, 자신의 Odometry 및 EBIMU yaw 데이터를 이용하여 마스터 경로를 추종한다. 이 과정은 마스터와 슬레이브 사이의 상대 위치를 계산하고 거리 오차와 방향 오차를 줄이기 위한 제어 명령을 생성하는 과정이다.

먼저 EBIMU는 슬레이브의 yaw 값을 측정한다. yaw 값은 슬레이브가 현재 어느 방향을 바라보고 있는지를 나타내는 방향 정보이며, 슬레이브의 진행 방향 보정에 사용된다. 이후 슬레이브는 모터 회전량 또는 주행 기록계를 기반으로 Odometry를 계산하여 자신의 이동 거리와 위치 변화를 추정한다.

마스터에서 전송된 path 데이터는 슬레이브의 기준 좌표계와 정렬된다. 슬레이브는 자신의 현재 위치와 마스터 경로의 목표 지점 사이의 거리 오차 및 방향 오차를 계산한다. 거리 오차가 크면 전진 속도를 증가시키고 방향 오차가 발생하면 각속도 명령을 생성하여 좌우 방향을 보정한다.

최종적으로 Raspberry Pi 4는 계산된 선속도와 각속도를 `/cmd_vel` 명령으로 변환하고 MD400T 모터 드라이버에 전달한다. MD400T는 좌우 모터의 회전 속도를 조절하여 슬레이브가 마스터의 경로를 따라 이동하도록 제어한다.

-4. 슬레이브 초음파 장애물 안전 정지 과정

슬레이브는 USB Webcam을 통해 마스터에 부착된 ArUco 마커를 인식한다. 이 기능은 마스터와 슬레이브 간 Wi-Fi 통신이 불안정하거나, UDP 기반 위치 데이터가 일정 시간

이상 수신되지 않는 경우 보조 추종 방식으로 사용된다. PDF의 전체 동작 원리에서도 Wi-Fi 통신이 끊길 경우 카메라로 ArUco 마커를 인식하여 마스터를 추종하는 방식으로 전환한다고 설명되어 있다.

USB Webcam은 실시간 영상 데이터를 수집하고 영상 내에서 ArUco 마커 ID를 탐지한다. 마커가 인식되면 슬레이브는 마커의 중심 좌표와 크기 정보를 기반으로 마스터와의 상대 거리 및 좌우 방향 오차를 계산한다. 마커가 화면 중앙보다 왼쪽에 있으면 좌측 방향 보정 명령을 생성하고 오른쪽에 있으면 우측 방향 보정 명령을 생성한다.

마커와의 거리가 너무 멀 경우 슬레이브는 전진 명령을 생성하고 목표 거리 이내로 접근하면 정지한다. 이때 마커 중심 좌표의 좌우 오차는 각속도 제어에 사용되고 마커와의 거리 오차는 선속도 제어에 사용된다.

ArUco 마커가 카메라 시야에서 사라지거나 인식되지 않을 경우, 슬레이브는 잘못된 방향으로 이동하지 않도록 `/cmd_vel` 값을 0으로 변경하여 정지한다. 이를 통해 통신 장애 상황에서도 최소한의 안전성을 확보한다.

-5. 슬레이브 초음파 장애물 안전 정지 과정

슬레이브는 초음파 센서 4개를 사용하여 주변 근거리 장애물을 감지한다. 초음파 센서는 Echo/Trig 신호를 이용해 장애물까지의 거리를 측정하며, 측정된 거리 값은 Raspberry Pi 4로 전달된다.

슬레이브는 초음파 센서에서 측정한 거리 값을 기준으로 장애물 존재 여부를 판단한다. 장애물이 50cm 이내에서 감지되면 슬레이브는 현재 추종 명령이나 전진 명령을 무시하고 정지 명령을 우선 적용한다. 장애물이 50cm 이내로 더 가까워지는 경우에는 충돌 위험이 높다고 판단하여 정지 상태를 유지하고 55cm 이상이 되었을 시에 다시 재주행한다.

이 과정에서 초음파 기반 정지 명령은 Odometry + IMU 추종 명령이나 ArUco 마커 추종 명령보다 높은 우선순위를 가진다. 즉, 슬레이브가 마스터 경로를 정상적으로 추종하고 있더라도 장애물이 감지되면 즉시 모터 구동을 중단한다.

장애물이 제거되면 슬레이브는 다시 센서 값을 확인하고 안전 거리가 확보된 경우에만 추종 주행을 재개한다. 이를 통해 슬레이브 카트가 보행자, 수하물, 벽면, 기둥 등과

충돌하지 않도록 한다.

5 요약 및 결론

1. 프로젝트 개요 및 배경

목표 및 필요성: 다문화 사회 및 국제 교류 증가에 따른 수하물 운송 소유 폭증에 대응하하며 기존 공항 카트 및 자율주행 캐리어 로봇의 한계를 극복하기 위해 AI 기반 무선 추종 트랙커 개발을 목표로 삼음. 이는 공항 내 수하물 운송의 효율성과 안전성 향상에 기여할 것이다.

핵심 가치: 인프라 구축 없이 적용 가능한 무선 추종형 시스템으로서 IoT 및 머신러닝 기술 융합형 운송 자동화 솔루션으로서의 가치를 가진다.

기대효과: 저비용 구조로 대규모 운용이 가능하며 경로 탐색 정확도 및 장애물 회피 성능 향상을 기대함. 또한 고령자, 가족 단위 이용객의 편의성 개선 및 공항 운영비 절감 효과 예상

2. 요구사항 정의 및 기능 설계

2-1. 하드웨어 설계:

프레임 및 구동: 최대 적재 하중(20kg)을 견딜 수 있는 강성 확보 및 1kg 이상 화물 견인을 위한 DC Geared Motor(12V) 사용

제어 유닛: ROS2 Humble 구동을 위한 라즈베리파이 5 모델 B (4GB RAM) 선정

센서: SLAM/AMCL 및 환경 인식을 위한 360도 스캔 가능 2D 라이더 센서 장착, 정밀 주행을 위한 IMU 센서 탑재, 추종 제어를 위한 라이더 또는 카메라 탑재

안전: 물리적 비상 정지 스위치 탑재

2-2. 소프트웨어 설계:

마스터 기능 (S-M): ROS2 Nav2를 사용한 SLAM/AMCL 기반 자율주행 및 경로 생성 수행(10Hz 이상), 장애물 감지 시 슬레이브에게 즉시 정지 명령 전송, ROS2 네임스페이스 기반 다중 슬레이브 관리

슬레이브 기능 (S-S): 마스터로부터 속도/경로 명령 수신 및 주행 제어 수행, 전방 초음파/ToF 센서를 이용한 **PID 제어**로 목표 추종 거리(1.0m 허용 오차) 유지, 통신 타임아웃 발생 시 안전 정지 모드로 자동 전환

네트워크 기능 (S-N): ROS2 DDS 기반 Wi-Fi 무선 통신, UDP(Best Effort)를 통한 실시간 제어 데이터 멀티캐스팅으로 지연 최소화, SROS2를 통한 비승인 장치 접근 제어

3. 성능 요구사항:

주행 성능: 평상시 1.0 ~ 1.2 m/s 주행 속도 유지, 최대 20kg까지 견인 가능

정밀도: 직선 주행 시 10cm 이내 오차 유지

실시간성: End-to-End 통신 지연 100ms 이내 확보

안전: 1.0 m/s 주행 중 정지 명령 시 0.5m 이내 완전 정지

4. 결론

본 졸업작품 설계서는 공항 및 대형 물류 거점에서 증가하는 수하물 운송 수요와 인력 부담을 완화하기 위해, ROS2 기반 무선 추종 트롤리 시스템의 개념을 정립하고 하드웨어, 소프트웨어, 네트워크 관점에서 통합 설계를 수행하는 것을 목표로 하였다. 사용자를 일정 거리에서 따라가며 이동하고 장애물을 회피하면서

안전하게 주행할 수 있는 자율주행 트롤리를 설계함으로써 향후 공항·대형 마트·물류센터 등에서의 자동화 가능성을 모색하였다.

하드웨어 측면에서는 최대 20kg 하중을 견딜 수 있는 트롤리 프레임과 구동 모터, 360° 라이다 센서를 이용한 환경 인식, IMU 및 ToF 센서를 이용한 정밀 추종 제어, 비상 정지 스위치 등 안전 장치를 포함하는 구조를 정의하였다. 이를 통해 실제 운용 환경에서 요구되는 안정성, 하중 조건, 안전 규격을 만족할 수 있도록 기계적 요소를 체계적으로 설계하였다.

소프트웨어 및 네트워크 측면에서는 ROS2를 기반으로 한 마스터–슬레이브 이중 구조를 설계하였다. 마스터 노드는 SLAM/AMCL을 이용한 지도 작성과 경로 생성, 장애물 회피 알고리즘을 담당하였고 슬레이브 노드는 마스터로부터 전달된 속도 및 경로 명령을 받아 PID 제어를 수행하며 목표 추종 거리를 유지하도록 하였다. 또한 Wi-Fi 기반 ROS2 DDS 통신과 UDP(베스트 에포트) 방식을 활용하여 실시간 데이터 교환을 전제로 한 네트워크 구조와 토픽 구성을 설계하였다. 설계 단계에서 지연 시간, 패킷 손실, 안전 모드 전환 등의 시나리오를 고려하여 토폴로지와 메시지 구조를 정의했다는 점에 의의가 있다.

성능 측면에서는 실제 구현 대신, 설계 단계에서 달성하고자 하는 목표 성능 요구사항을 명확히 설정하였다. 대표적으로 일정 무게를 제한을 넣어 안정적인 주행을 하도록 하는 주행 성능 판단, 직선 주행 시 1.0m 이내의 위치 오차를 목표로 추종 제어 파라미터 정의 정밀도 정의, 마스터-슬레이브 간의 네트워크 구조 설계에

의한 실시간성, 비상정지 및 감속 시나리오 고려 등 안전성 설계로 성능에 기준을 두었다.

프로젝트를 진행하며, 단순히 개별 기술을 공부하는 것과 달리 하나의 시스템을 처음부터 끝까지 설계한다는 것의 어려움과 보람을 동시에 느낄 수 있었다. 요구사항 정의 단계에서는 “실제 공항에서 정말 쓸 수 있을까?” 라는 관점에서 하중, 속도, 안전 규격을 고민해야 했고, 아키텍처 설계 단계에서는 LIDAR, IMU, ToF, Wi-Fi 통신 등 다양한 요소를 한 시스템 안에서 어떻게 연결할지 끊임없이 트레이드오프를 검토해야 했다. 특히 ROS2 기반 마스터–슬레이브 구조를 설계하면서, 토픽 구조, QoS 설정, 장애 상황에서의 복구 절차까지 고려해야 한다는 점을 통해 로봇 시스템 통합의 복잡성과 책임감을 크게 체감할 수 있었다.

또한 이번 졸업작품 설계서를 작성하면서 실제 구현 과정에 앞서 필요한 이론 설계, 요구사항 정의, 아키텍처 구상, 인터페이스 명세를 체계적으로 정리했다는 점에서 의미가 컸다. 앞으로 이 설계서를 기반으로 프로토타입을 구현하고, 센서 융합 알고리즘 및 제어 파라미터를 튜닝하며, 실제 환경에서 성능을 측정한다면 설계 단계에서의 가정이 얼마나 유효했는지 검증할 수 있을 것이다.

향후 발전 방향으로는

- UWB 등 추가 센서를 활용한 정밀 위치 측위 및 추종 안정성 향상,
- 배터리 용량 및 모터 스펙 최적화를 통한 운용 시간 및 하중 성능 개선,

- 복수 트롤리간 협업을 위한 다중 로봇 플릿 관리 기능,
- 공항·병원·물류 센터 등 실제 현장 시나리오를 반영한 HMI(사람-로봇 상호작용) 설계등이 있다. 이러한 후속 연구와 구현 단계를 통해, 본 설계서에서 제안한 무선 추종 트롤리 시스템이 실제 산업 현장에서 활용 가능한 수준의 플랫폼으로 발전하길 기대한다.

최종적으로 본 졸업작품 설계는 무인 운송 로봇의 요구사항 분석부터 하드웨어, 소프트웨어, 네트워크 통합 설계까지 전 과정을 경험하게 해 준 의미 있는 작업이었다. 설계를 진행하면서 로봇공학, 제어, 네트워크, IoT 등 수업에서 배운 내용을 실제 시스템 관점에서 다시 한번 정리할 수 있었고, 앞으로의 로봇 자율주행 관련 진로를 구체적으로 고민해 보는 계기도 되었다. 이 설계서가 향후 구현 단계로 이어지는 기반 문서이자, 향후 발전 가능성을 담은 청사진이 되기를 바라는 바이다.

부록 1: 참고 문헌

[1] 김시찬, 구현우, 유건민, 이현서, 황성수, “ROS2 및 Nav2를 활용한 단안 카메라 기반 실내 자율주행 로봇 시스템 구현,” 2025 한국컴퓨터종합학술대회 논문집, pp. 1985-1987, 2025.

[2] https://docs.nav2.org/

[3] https://www.youtube.com/watch?v=s2UhBFxE_e8

[4] http://youtube.com/watch?v=SMZCio24Zx4&t=3s

부록 2: 용어

용어	정의
H-S	Hardware-Slave

H-M	Hardware-Master
H-P	Hardware-Performance
S-S	Software-Slave
S-N	Software-Network
S-M	Software-Master
PR	Performance
TC	Test Case
TC-SYS	Test Case-System
SLAM	Simultaneous Localization and Mapping